

General Instruction

1. Student Instruction

Basic Rules

1. Each team must consist of 1–4 students.
2. Every team must submit via GitHub Repository Link
 - Working Project (Frontend + Backend)
 - Final Presentation (PPT and Demo)
3. Project must be:
 - Original
 - Developed during the hackathon timeline
4. AI tools (ChatGPT, Copilot, etc.) are allowed for guidance, but:
 - You must understand your code.
 - You must explain your logic during judging.

Timeline Discipline

- No late submissions.
- Final Git commit time will be verified.
- Last-minute ZIP submissions without proper repository history will be rejected.

Project Expectations

Your project must include:

- Working UI
- Proper Backend/API
- Database integration
- Deployment (recommended and Preferable)

2. Git Repository & Team Development Rules

Mandatory Requirements

1. Repository must be created by the team leader within 24 hours of releasing the problem statement.
2. Repository name format: NavKalpana-TeamCode
Example: NavKalpana-RICR-NK-0001
3. All team members must:
 - Must be the contributor in repo
 - Use individual GitHub accounts
 - Contribute via commits
 - Avoid single-person commits for team projects

Commit Rules

- Minimum 10 meaningful commits.
- Avoid a single bulk commit at the end.
- Commit messages must be clear.
 - Good Commit Example:
 - Added login authentication
 - Integrated payment API
 - Fixed cart validation bug

- Bad Commit Example:

- Update
- Final
- Added
- done

Bad Commit Messages will lead to negative marking

Branching Rule (Recommended)

- main – Final stable version
- All the Members must Create Separate Branches which further merges to the Main Branch
 - Example:
 - feature/login
 - feature/register
 - bug/update etc.

3. Recommended Project Folder Structure

Repository Name Example: *NavKalpana-RICR-NK-0001*

Standard Structure

- ❖ NavKalpana-TeamCode
 - frontend
 - backend
 - docs
 - problem-statement.pdf
 - architecture-diagram.png
 - api-documentation.md
 - presentation.pptx
 - README.md

4. README.md Must Include

- Project Title
- Team Members & Roles
- Problem Statement
- Tech Stack Used
- Installation Steps
- API Endpoints
- Screenshots
- Future Improvements

5. What Judges Will Verify

Technical Verification

- Git commit history
- Contribution by all members
- Code understanding

- Folder structure clarity
- Backend API functionality
- Database integration
- Error handling

Innovation Check

Judges will evaluate:

- Whether it solves a real problem
- Practical usability
- Scalability potential
- Meaningful use of AI (if included)

Viva Round Questions

Each member may be asked:

- Explain your module.
- Explain database schema.
- Explain API flow.
- Explain deployment steps.
- Explain one technical challenge you faced.

6. Important Warning

Immediate disqualification may occur if:

- Code is copied from the internet without modification
- Purchased templates are used without proper logic implementation
- Repository is created only on the final day
- Only UI is presented without functionality
- Fake data is shown without backend logic (unless clearly defined prototype)
- Only one member answers all questions

7. Bonus Considerations

- Clean UI/UX
- Proper Authentication
- Role-based access control
- AI/ML Integration (if required)
- Deployment (Vercel, Render, AWS, Azure)
- Clean code practices

Final Note

The hackathon is not only about coding. It is about teamwork, structured thinking, problem solving, and clear communication. Build a meaningful solution and be prepared to explain it confidently.

RideShareX

Intelligent Ride Sharing & Carpooling Platform

1. Product Overview

RideShareX is a web-based ride-sharing and carpooling platform that connects drivers and passengers traveling on similar routes, enabling safe, affordable, and structured shared transportation.

The system operates as a structured transaction loop:

Profile → Offer/Search Ride → Match → Book → Confirm → Travel → Rate → Repeat

Phase 1 focuses on secure ride matching, booking lifecycle, communication, and trust mechanisms.

2. Product Vision

To build a reliable, trust-driven, scalable ride-sharing ecosystem that:

- Reduces travel costs
- Encourages shared mobility
- Ensures safety & transparency
- Enables efficient ride matching
- Builds user trust through reputation

3. System Architecture

RideShareX consists of:

PART 1 – Core Ride Matching & Booking Engine (~70%)

PART 2 – Trust Intelligence & Growth Optimization Layer (~30%)

PART 1 – CORE RIDE MATCHING & BOOKING ENGINE (70%)

This layer handles:

User onboarding → Ride listing → Search → Booking → Confirmation → Communication → Completion

4. Authentication & Identity Module

Features

Email / Phone signup

Password hashing (bcrypt / Argon2)

JWT-based sessions

Logout & session timeout

OTP verification (phone/email)

Identity Validation (Basic Trust Layer)

Profile photo required

Phone verification mandatory

Optional ID verification badge

5. User Profile System

5.1 Driver Profile

Full name
Profile photo
Phone (verified)
Vehicle details
Short bio
Ratings
Verification status

5.2 Passenger Profile

Full name
Profile photo
Phone (verified)
Ratings
Ride history

6. Ride Creation Module (Driver)

Drivers can post a ride by entering:

Departure location
Destination location
Optional route stops
Date & departure time
Available seats
Price per seat
Vehicle details
Luggage allowance
Ride preferences (no smoking, pets allowed, etc.)

Ride Status Lifecycle

Active
Fully Booked
Cancelled
Completed

7. Ride Discovery & Search Engine (Passenger)

Search Inputs

From location
To location
Travel date
Seats required

Filtering Options

Departure time
Price range
Available seats

Driver rating
Instant confirmation only

Sorting Options

Lowest price
Earliest departure
Best rated driver

8. Matching & Booking Engine

Booking Flow

Passenger selects ride
Selects seats
Sends booking request
Driver receives notification
Driver confirms or rejects

Booking Status Flow

Requested
Accepted
Rejected
Cancelled
Completed

Seat Management Logic

Seats available reduce automatically after confirmation
If seats reach zero → Ride marked Fully Booked

9. Messaging & Communication Module

Secure in-app messaging between:

Driver ↔ Passenger

Features:

Chat inbox

Message history

Unread indicators

Notifications

Messages enabled only after booking request to prevent spam.

10. Trip Completion & Rating System

After trip date passes:

Ride marked as Completed

Both users prompted to rate

Rating System:

1–5 stars

Optional comment

Displayed on profile.

11. Notification Engine

Triggered when:

Ride posted

Booking requested

Booking accepted

Booking rejected

Message received

Ride cancelled

Trip completed

Delivery via:

In-app notification

Email (optional)

SMS (optional)

12. Core Dashboard

Driver Dashboard

Upcoming rides

Booking requests

Completed rides

Earnings summary (if commission model)

Rating overview

Passenger Dashboard

Upcoming trips

Booking history

Ride status

Saved routes

Rating summary